On Geometric Invariants, Learning, and Recognition of Shapes and Forms

Gautam Pai, Mor Joseph-Rivlin, Ron Kimmel and Nir Sochen

Abstract Extracting meaningful representations from geometric data has prime importance in the areas of computer vision, computer graphics, and image processing. Classical approaches use tools from differential geometry for modeling the problem and employed efficient and robust numerical techniques to engineer them for a particular application. Recent advances in learning methods, particularly in the areas of deep-learning and neural networks provide an alternative mechanism of extracting meaningful features and doing data-engineering. These techniques have proven very successful for various kinds of visual and semantic cognition tasks achieving state-of-the art results. In this chapter we explore the synergy between these two seemingly disparate computational methodologies. First, we provide a short treatise on geometric invariants of planar curves and a scheme to discover them from data in a learning framework, where the invariants are modelled using neural networks. Secondly, we also demonstrate the reverse, that is, imputing principled geometric invariants like geometric moments into standard learning architectures enables a significant boost in performance. Our goal would not only be to achieve better performance, but also to provide a geometric insight into the learning process thereby establishing strong links between the two fields.

Gautam Pai
Faculty of Computer Science, Technion, e-mail: paigautam@cs.technion.ac.il

Mor Joseph-Rivlin
Faculty of Electrical Engineering, Technion, e-mail: mor1joseph@campus.technion.ac.il

Ron Kimmel
Faculty of Computer Science, Technion, e-mail: ron@cs.technion.ac.il

Nir Sochen
Department of Applied Mathematics, Tel-Aviv University, e-mail: sochen@post.tau.ac.il

1 Introduction

Geometric invariance is an important issue in various problems of geometry processing, computer vision and image processing, having both a theoretical and a computational aspect to it. In order to solve problems like shape correspondence and retrieval, it is essential to develop representations which do not change under the action of various types of transformations. The difficulty of estimating invariants is usually related to the complexity of the transform for which the invariance is desired and the numerical construction of invariant functions from discrete sampled data is typically non-trivial.

Learning methods involve the study and design of algorithms that develop functional architectures which can be trained to perform a specific task when they are endowed with an objective cost function and a sufficient amount of data. Recent advances in convolutional neural networks and deep learning have catapulted their use in solving a multitude of problems in computer vision, speech processing, pattern recognition and many other data-sciences. Their success has been largely attributed to the meaningful semantic features they generate from the data which enables them to perform tasks like recognition, classification, discrimination etc. with state-of-the-art superiority.

This chapter provides a synergy between the principled computation of numerical geometry and the learning based schemes of neural networks and deep learning. In this chapter, we aim to explore and highlight possible benefits of using learning to enable better estimation of geometric invariants as well as using principled geometric constructions to enable superior learning. By bringing these two disparate themes together we hope to advocate the research for discovery of novel methods that can combine the valuable insight and understanding provided by principled axiomatic methods in geometry with the black-box-yet superior data dependant models of deep learning. To this aim, we make two such demonstrations in this chapter. First, in section 2 we design a setup to *learn* the curvature (specifically an invariant representation) of a curve from examples. A first course on differential geometry shows that the arc-length and curvature are fundamental invariants of a curve and provably invariant to specific class of transformations [1, 7, 6, 35]. Hence, the Euclidean arc-length and curvature are the simplest invariants to Euclidean transformations, Affine arc-length and affine curvature to affine transformations [29], and so on. We demonstrate experimentally that, by synthetically generating examples of these transformations, we can train a neural network to *learn* these invariants [24] from data. Importantly, we demonstrate that such invariants have superior numerical properties in terms of a much improved robustness to noise and sampling as compared to their axiomatic counterparts.

Second, in section 3 we demonstrate that the input of geometric moments into neural network architectures well known for point cloud processing result in an improved shape recognition performance. Classically, geometric moments have been used to define invariant signatures for classification of rigid objects [19]. Comparing two point clouds by correlating their shape moments is a well known method in the geometry processing literature. At the other end, geometric moments are composed

of high order polynomials of the points coordinates, and approximation those polynomial is not trivial to a neural networks [40]. Here, we propose to add polynomial functions of the coordinates to the network inputs, which will allow the network to learn the elements of the moments and should better capture the geometric structure of the data.

2 Learning Geometric Invariant Signatures For Planar Curves

2.1 Geometric Invariants of Curves

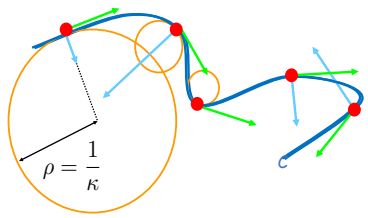
Invariant representations or invariants are functions defined over geometries which do not change despite the action of a certain type of transformation. These invariants are essential to facilitate our understanding of the idea of a shape. Kendall [18] provides a very apt definition: shape is what remains after translation, rotation and scale have been factored out. In general invariant theory, this can be extended to any general Lie group of transformation. Hence, we can state that shape is what remains after the effects of a certain type of deformation is factored out. Importantly, geometric invariants provide a means to characterize the shape and hence their efficient and robust computation is essential to the success of a given geometric processing framework.

In the next two subsections, we enumerate on two different types of invariants for the simplest geometric structures: planar curves. We discuss their theoretical properties and also numerical considerations for their efficient implementation. We show that the computational schematic involving these invariants correlates very aptly to convolutional neural networks. Therefore, by following the same overall computational pipeline, and replacing the key structural components with learnable options like convolutional kernels, and rectified linear units, we can learn the invariants from examples in a metric learning framework.

2.1.1 Differential Invariants

A classic theorem from E. Cartan and Sophus Lie [1], [9] characterize the space of invariant signatures for two and three dimensional geometric entities. For planar curves, it begins with the concept of invariant arc-length, which is a generalized notion of length of the curve. Given a particular Lie-group transformation, one can define an invariant arc-length and an invariant curvature with respect to this arc-length. The set of differential invariant signatures comprises of this invariant curvature and its successive derivatives with respect to the invariant arc-length [6], [7].

The basic idea is that, one can formulate a differential notion of length and a local signature, based on identifying invariance properties of a particular group of transformations. Consider the simple case of euclidean invariants as demonstrated



$$s = \int_{p=0}^{p=p_0} \sqrt{x_p^2 + y_p^2}\, dp \quad \kappa(p) = \frac{x_p\, y_{pp} - y_p\, x_{pp}}{(x_p^2 + y_p^2)^{\frac{3}{2}}}$$

Fig. 1: Schematic for differential invariants: Euclidean arclength s and Euclidean curvature κ which is the inverse of the radius of curvature of the osculating circle at a given point.

in Figure 1. Consider $C(p) = \begin{bmatrix} x(p) \\ y(p) \end{bmatrix}$: a planar curve with coordinates x and y parameterized by some parameter p. Locally at any given point p on the curve, we have the euclidean arclength given by:

$$ds = \sqrt{dx^2 + dy^2} = \sqrt{\left(\frac{dx}{dp}\right)^2 + \left(\frac{dy}{dp}\right)^2}\ dp \tag{1}$$

It is straightforward to observe that expression 1 is provably invariant to any euclidean transformations of the curve. Therefore, the length of a curve from point p_0 to p_1 is given by integrating the differential lengths:

$$s = \int_{p_0}^{p_1} |C_p|\, dp = \int_{p_0}^{p_1} \sqrt{x_p^2 + y_p^2}\, dp, \tag{2}$$

where $x_p = \frac{dx}{dp}$, and $y_p = \frac{dy}{dp}$. Similarly, a local signature at a point on the curve can be obtained by observing that the rate of change of the unit tangent vector with respect to the arc-length is invariant to euclidean transformations. This gives us the principal invariant signature, that is the Euclidean curvature, given by

$$\kappa(p) = \frac{\det(C_p, C_{pp})}{|C_p|^3} = \frac{x_p y_{pp} - y_p x_{pp}}{(x_p^2 + y_p^2)^{\frac{3}{2}}}. \tag{3}$$

In addition, the euclidean curvature of equation 3 is also the inverse of radius of the osculating circle at the point as shown in Figure 2.

A similar philosophy can used for the derivation of arc-length and curvature for the equiffine case: $\tilde{C} = A\,C + t$ with $\det(A) = 1$. We refer the interested reader to [29] for a comprehensive and detailed treatise on affine differential geometry. We have the differential equi-affine arc-length given by

$$dv = |\kappa|^{\frac{1}{3}}\ |C_p|\ dp \tag{4}$$

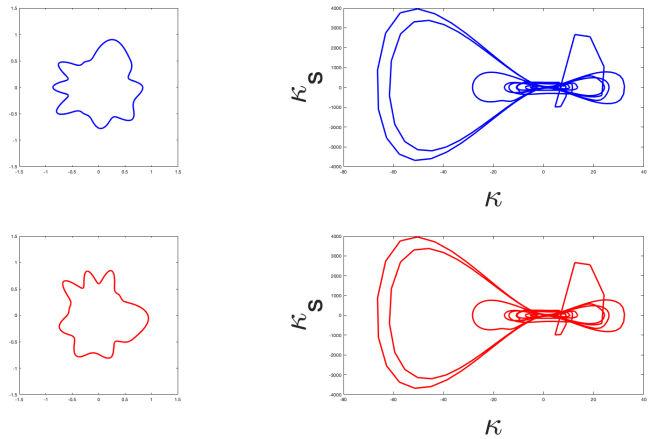


Fig. 2: Cartans theorem demonstrating differential invariant signatures. The curves (red and blue) are related by a Euclidean transformation. The plot of the first two differential signatures: $\{\kappa, \kappa_s\}$ are identical.

and hence the equi-affine length between points p_0 and p_1:

$$v = \int_{p_0}^{p_1} |\kappa|^{\frac{1}{3}} \, |C_p| \, dp, \tag{5}$$

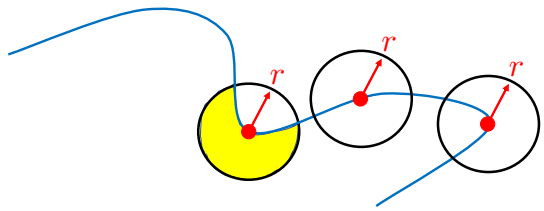
and the equi-affine curvature,

$$\mu = (C_{vv}, \; C_{vvv})$$
$$\mu = \frac{4(x_{pp}y_{ppp} - y_{pp}x_{ppp}) + (x_p y_{pppp} - y_p x_{pppp})}{3(x_p y_{pp} - y_p x_{pp})^{\frac{5}{3}}} - \frac{5(x_p y_{ppp} - y_p x_{ppp})^2}{9(x_p y_{pp} - y_p x_{pp})^{\frac{8}{3}}} \tag{6}$$

These differential invariant signatures are unique, that is, two curves are related by the corresponding transformation *if and only if* their differential invariant signatures are identical. Moreover, every invariant of the curve is a function of these fundamental differential invariants and its successive derivatives with respect to the invariant arc-length. Thus, we have the set of euclidean differential invariants: $S_{euclidean}$, and the set of equi-affine differential invariants: $S_{equi-affine}$, for every point on the curve:

$$S_{euclidean} = \{\kappa, \; \kappa_s, \; \kappa_{ss}, \; ...\} \tag{7}$$
$$S_{equi-affine} = \{\mu, \; \mu_v, \; \mu_{vv}, \; ...\} \tag{8}$$

Cartan's theorem provides a strong characterization of the invariant signatures and establishes invariant curvature as the principal signature of a curve. For planar curves, any contour can be recovered up to the transformation given only the first



$$I_r(p) = \int_{\mathcal{C}} h_r(p,x)\, d\mu(x)$$

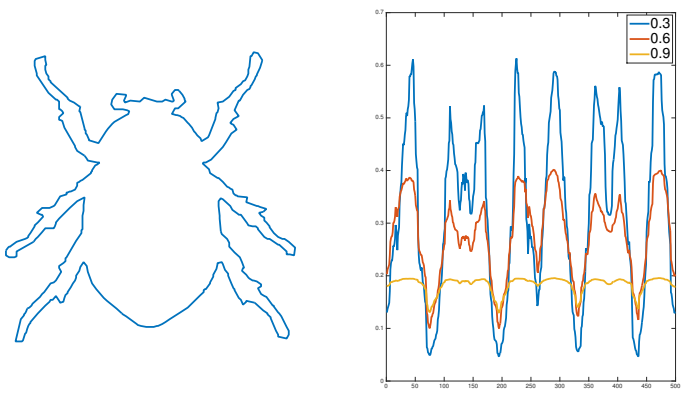


Fig. 3: (top) Schematic for integral invariants: the integral area invariant. At each point, for a predefined radius r, the signature is evaluated as the area of the yellow region. (bottom) Integral invariant signatures plotted for a closed contour for different raddii of the ball. Each signature captures structure at a different scale as defined by the radius of the ball.

two sets of invariants, that is $\{\kappa, \kappa_s\}$, for euclidean, and $\{\mu, \mu_v\}$ for the equi-affine. Figure 2 demonstrates this phenomenon.

An important observation is that differential invariants are, as the name implies, *differential* in nature, that is, the formulas involving their computation involve derivatives of the curve. From a signal processing perspective, this is equivalent to *high-pass filtering* over the coordinate functions of the curve combined with further non-linearities. Therefore, numerical estimation of these invariants is a challenge in low signal-to-noise ratio settings.

2.1.2 Integral Invariants

Integral invariants are functions computed using integral measures along the curve using parameterized kernels $h_r(p,x) : \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$

$$I_r(p) = \int_C h_r(p, x) d\mu(x) \tag{9}$$

In contrast to differential invariants, integral invariants are constructed by designing invariant functions that are a result of local integral computations on the curve. Typically, integral invariants are associated with a parameter r that characterizes the extent of the integration locally at each point. Typical examples are the integral distance invariant for which $h_r(p, x)$ is the distance function of the point p to all points on the curve within a ball of radius r. Another more prominent example is the integral area invariant which, for each point on the contour, computes the area of the intersection between a ball of radius r and the interior of the curve. [21], [11], [25], [13] provide a detailed treatise on various kinds of integral invariants and their theoretical and numerical properties.

In contrast to differential invariants, integral invariants are more robust to noise due to the integral nature of their computation. Moreover, The radius r provides a scale-space like property, where a big enough r can be set to compensate for all noise within that magnitude. Figure 3 shows an example of Integral Invariants.

2.2 Learning Geometric Invariant Signatures of Planar Curves

2.2.1 Motivation

It is straightforward to observe that differential and integral invariants can be thought of as being obtained from non-linear operations of convolution filters. The construction of differential invariants employ filters for which the action is equivalent to numerical differentiation (high pass filtering) whereas integral invariants use filters which act like numerical integrators (low pass filtering) for stabilizing the invariant. This provides a motivation to adopt a learning based approach and we demonstrate that the process of estimating these filters and functions can be outsourced to a learning framework.

We use the Siamese configuration (see Figure 4) for implementing this idea. Siamese configurations have been used in signature verification [4], face-verification and recognition [31], [32], [14], metric learning [10], image descriptors [8], dimensionality reduction [12] and also for generating 3D shape descriptors for correspondence and retrieval [22], [38]. In these papers, the goal was to learn the descriptor and hence the similarity metric from data using notions of only positive and negative examples. We use the same framework for estimation of geometric invariants. However, in contrast to these methods, we also discuss and analyze the output descriptor and provide a geometric context to the learning process. The contrastive loss function (equation 10) driving the training ensures that the network chooses filters which push and pull different features of the curve into the invariant by balancing a mix of robustness and fidelity.

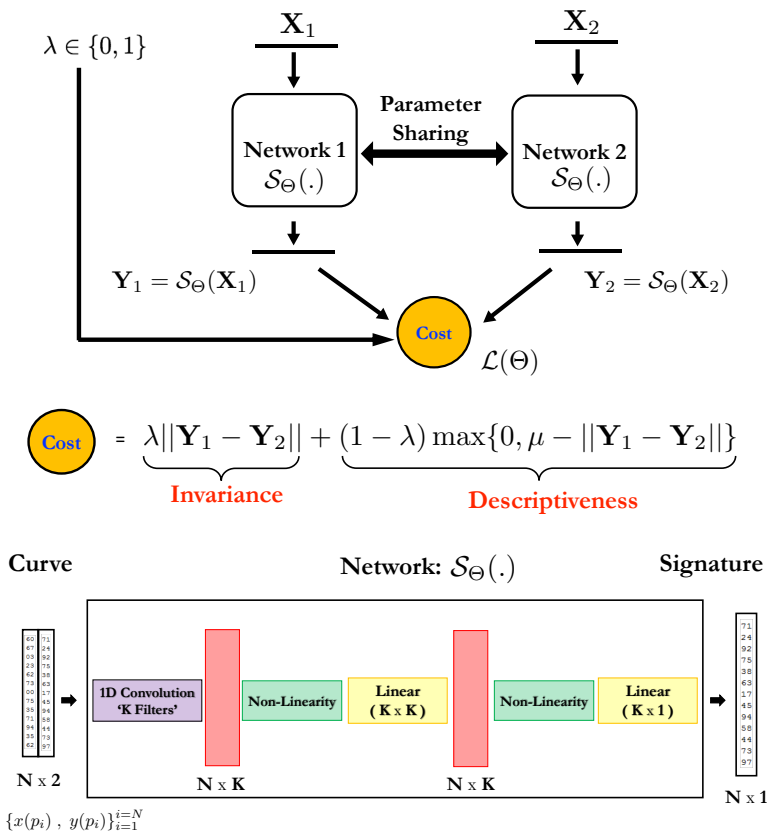


Fig. 4: (top) Siamese configuration for learning an invariant signature of the curve. The cost function comprises of two components: positive examples to train for invariance and negative examples to train for descriptiveness. (bottom) An example of a straightforward one dimensional convolutional neural network architecture that inputs a curve and outputs a point-wise signature of the curve.

2.2.2 Training for Invariance

A planar curve can be represented either explicitly by sampling points on the curve or implicitly using a representation like level sets [19]. We work with an explicit representation of simple curves (open or closed) with random variable sampling of the points along the curve. Thus, every curve is a $N \times 2$ array denoting the 2D coordinates of the N points. We build a convolutional neural network which inputs a curve and outputs a representation or signature for every point on the curve (see (bottom) in Figure 4). We can interpret this architecture as an algorithmic scheme of representing a function over the curve.

A Siamese configuration (see figure 4) for training neural networks comprises of stacking two identical copies of a network (hence shared parameters as shown in Figure 4), that process two separate inputs of data. A training example comprises

of a pair of data points ($\mathbf{X_1}$ and $\mathbf{X_2}$ in Figure 4) and a label: λ, which deems them as a positive ($\lambda = 1$) or a negative ($\lambda = 1$) example. During training, for each pair of positive examples ($\lambda = 1$), the loss forces the network to adjust its parameters so as to reduce the difference in signatures for this pair. For each pair of negative examples ($\lambda = 0$), the loss tries to maximise the the difference in signature outputs of the network. We extract geometric invariance by requiring that the two arms of the Siamese configuration minimize the distance between the outputs for curves which are related by a transformations and maximize for carefully constructed negative examples. For a detailed discussion on the choice of negative examples, [24] provides an explanation that connects invariant curve-evolution and multiscale curve descriptors to the proposed learned signatures.

The intuition behind this positive-negative example based contrastive training is as follows. Training with only positive examples will yield a trivial signature like an all-ones for every curve irrespective of its content. Hence training with negative examples $\lambda = 0$ enables the injection of a sense of descriptiveness into the signature. Minimizing the contrastive cost function (Equation 10) of the Siamese configuration directs the network architecture to model a function over the curve which is invariant to the transformation and yet descriptive enough to encapsulate the salient features of the curve.

We build a sufficiently large dataset comprising of such positive and negative examples of the transformation from a database of curves. Let $\mathbf{X_1}^{(i)}, \mathbf{X_2}^{(i)} \in \mathbb{R}^{2\times n}$ be two n-point curves inputted to the first and the second arm of the configuration for the ith example of the data with label λ_i. Let $\mathcal{S}_\Theta(\mathbf{X})$ denote the output of the network for a given set of weights Θ for input curve $\mathbf{X}$. The contrastive loss function is given by,

$$\begin{aligned}\mathcal{L}(\Theta) = \sum_i \; & \lambda_i \; || \; \mathcal{S}_\Theta(\mathbf{X_1}^{(i)}) - \mathcal{S}_\Theta(\mathbf{X_2}^{(i)}) \; || \\ & + \; (1 - \lambda_i) \; \max\Big(0, \; \mu \; - \; || \; \mathcal{S}_\Theta(\mathbf{X_1}^{(i)}) - \mathcal{S}_\Theta(\mathbf{X_2}^{(i)}) \; ||\Big) \qquad (10)\end{aligned}$$

where μ is a cross validated hyper-parameter, known as margin, which defines the metric threshold beyond which negative examples are penalized. Minimizing the contrastive cost function of the Siamese configuration directs the network architecture to model a function over the curve which is invariant to the prevailing transformation.

2.2.3 Results

Figure 5 and 6 visualize the output of the learned invariant. To test the numerical stability and robustness of the invariant signatures we add increasing levels of zero-mean Gaussian noise to the curve and compare the three types of signatures: differential (euclidean curvature), integral (integral area invariant) and the output of our network (henceforth termed as network invariant) as shown in figure 5. The network invariant shows a much improved robustness to noise in comparison to its

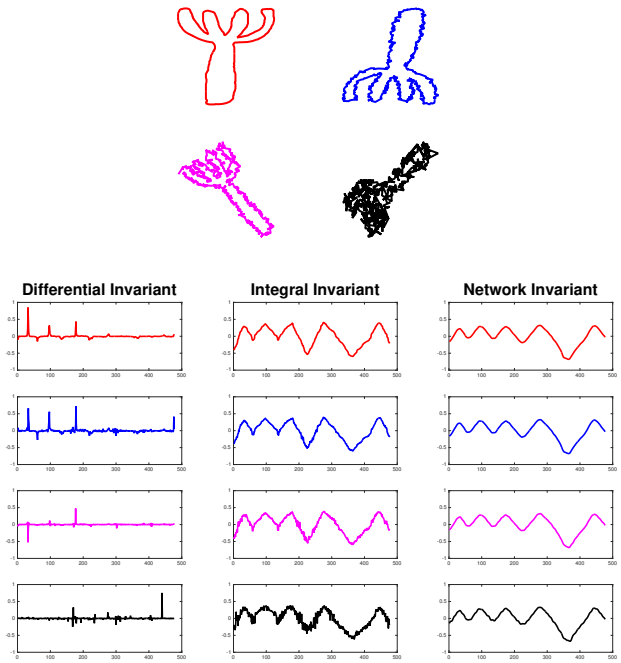


Fig. 5: Stability of different signatures in varying levels of noise and Euclidean transformations. The correspondence for the shape and the signature is the color. All signatures are normalized.

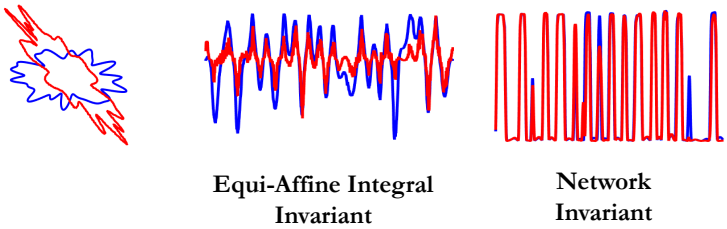


Fig. 6: **Equi-affine Invariants:** A simple smooth curve, equi-affine transformed along with a small addition of noise. The network is able to learn a much more robust invariant.

axiomatic counterparts. Apart from adding noise, we also transform the curve to obtain a better assessment of the invariance property.

The results demonstrate that deep learning architectures can indeed be used for learning geometric invariants of planar curves from examples. We show that the learned invariants are more robust to noise in comparison to their axiomatic counterparts. More importantly, our experiments lay out a discussion that provides

two different viewpoints to the very important topic of geometric invariance: that of classical differential geometry and deep learning.

3 Geometric Moments for Advanced Deep Learning on Point Clouds

In the previous section we use a learning mechanism to train for a geometric invariant. We argue that the computational mechanism of differential and integral invariants caters well to a convolutional neural network architecture and demonstrate a framework to learn these invariants from examples. In the this section, we demonstrate the reverse, that is, we input geometric moments into a learning architecture and discuss the benefits of this approach.

In recent years the popularity and demand of 3D sensors has vastly increased. Applications using 3D sensors include robot navigation [34], stereo vision, and advanced driver assistance systems [39] to name just a few. Recent studies attempt to adjust deep neural networks to operate on 3D data representations for diverse geometric tasks. Motivated mostly by memory efficiency, our choice of 3D data representation is to process raw point clouds. An interesting concept is applying geometric features as input to neural networks acting on point clouds for the geometric task of rigid object classification. This is a fundamental problem in analyzing rigid bodies which their isometry group includes only rigid transformations.

3.1 Geometric Moments as Class Identifiers

Historically, *geometric moments have been used to define invariant signatures for classification of rigid objects* [5]. Geometric moments of increasing order represent distinct spatial characteristics of the object, implying a strong support for construction of global shape descriptors. By definition, first order moments represent the extrinsic centroid; second order moments measure the covariance and can also be thought of as moments of inertia. We restrict ourselves to point clouds that can be seen as a set of unstructured 3D points that approximate the geometry of the surface. A point cloud is a set of points $X \in \mathbb{R}^3$, where each point $\boldsymbol{x}_j$ is given by its coordinates $(x_j, y_j, z_j)^T$, the moments in that discrete case are defined in Eq.11. Note that in Eq.11b the sum is over matrices, while in Eq.11c the sum is over the point coordinates and only one $(p + q + r)^{th}$ moment is computed.

$$1^{st}\ Order\ Moment: \quad \boldsymbol{x}_0 = \sum_j \boldsymbol{x}_j \tag{11a}$$

$$2^{nd}\ Order\ Moment: \quad \Sigma = \sum_j \boldsymbol{x}_j \boldsymbol{x}_j^T \tag{11b}$$

$$(p+q+r)^{th}\ Order\ Moment: \quad m_{pqr} = \sum_j (x_j)^p (y_j)^q (z_j)^r \tag{11c}$$

Higher-order geometric moments provide us descriptors of the surface: taking more orders improves our ability to identify our object. Sampled surfaces, such as point clouds can be identified by their estimated geometric moments, see equation 11. Moreover, a finite set of moments is often sufficient as a compact signature that defines the surface. This idea was classically used for classification of rigid objects and estimation of surface similarity.

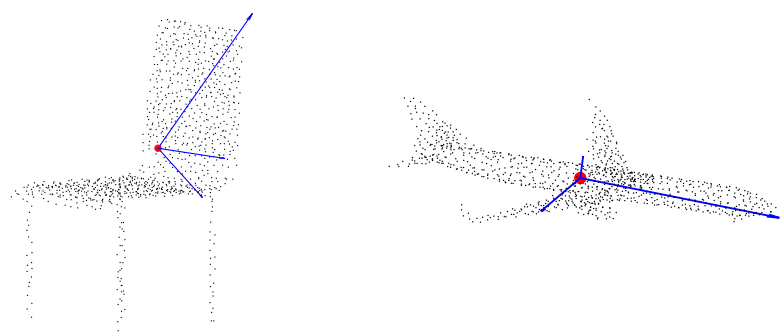


Fig. 8: The first and second geometric moments displayed on a point cloud. Using the first order moments (red disc), the translation ambiguity can be removed. The principal directions d_1, d_2, d_3 (blue arrows) are defined by the second order geometric moments.

Our goal is to allow a neural network to simply lock onto variations of geometric moments. One of the main challenges of this approach is that training a neural network to approximate polynomial functions requires the network depth and complexity to be logarithmically inversely proportional to the approximation error [40]. In practice, in order to approximate polynomial functions of the coordinates for the calculation of geometric moments the network requires a large number of weights and layers.

In the recent years an extended research has been done on processing point clouds [3, 27, 2, 33, 16, 15, 20] for object classification. Qi et al. [26] were the pioneers proposing a network architecture for directly processing point sets. The framework they suggested includes lifting the coordinates of each point into a high dimensional learned space, while ignoring the geometric structure. At the other end, networks that attempt to process other representations of low dimensional geometric structures such as meshes, voxels (volumetric grids) [23, 28, 36, 37] and multi-view projections

[17, 30, 41] are often less efficient when considering both computational and memory complexities.

3.2 Raw Point Cloud Classification based on Moments

Here, we propose a new layer that favors geometric moments for point cloud object classification. The most prominent element of the network is supplementing the given point cloud coordinates together with polynomial functions of the coordinates. This simple operation allows the network to account for higher order moments of a given shape. Explicitly, we simply add polynomial functions of the coordinates to the network inputs that will allow the network to learn the elements of Σ (see Eq.11) and, assuming consistent sampling, should better capture the geometric structure of the data.

First, we present a toy example to illustrate that extra polynomial expansions as an input can capture geometric structures well. Figure 10 (up) shows the data and the network predictions for 1000 noisy 2D points, each point belongs to one of two spirals. We achieved 98% success with extra polynomial multiplications, i.e x^2, y^2, xy, as input to a one layer network with only 8 hidden ReLU nodes, contrary to the same network without the polynomial multiplications (only 53% success). Figure 10 (down) shows the decision boundaries that were formed from the 8 hidden nodes of the network in Figure 10 (up left). As expected, radial boundaries can be formed from the additional polynomial extensions and are crucial to forming the entire decision surface.

Second, we present our approach that consumes directly point clouds with their second order polynomial expansions for object classification. An early attempt to treat point clouds as non-structured data sets from which a neural network is trained to extract discriminative properties is the PointNet [26] architecture proposed by Qi et al. The classification architecture of PointNet is based on fully connected layers and symmetry functions, like max pooling, to establish invariance to potential permutations of the points. The architecture pipeline commences with Multi-Layer Perceptrons (MLPs) to generate a per point feature vector, then, applies max pooling to generate global features that serve as a signature of the point cloud. Finally, fully connected layers produce output scores for each class. Note that all MLPs operations are performed per point, thus, interrelations between points are accomplished only by weight sharing.

The idea of integrating the coordinates of points in space into ordered vectors that serve as class identifiers motivated us to extend this line of thought in that of classification by comparing shape moments [5]. We claim that PointNet is an implicit way for approximating moments. With this fundamental understanding in mind, the next question is how could we assist the network by lifting it into a more convenient space. This observation allows us to use a classical simple lifting of the feature coordinates. It improves significantly both complexity and accuracy as validated in the experimental results.

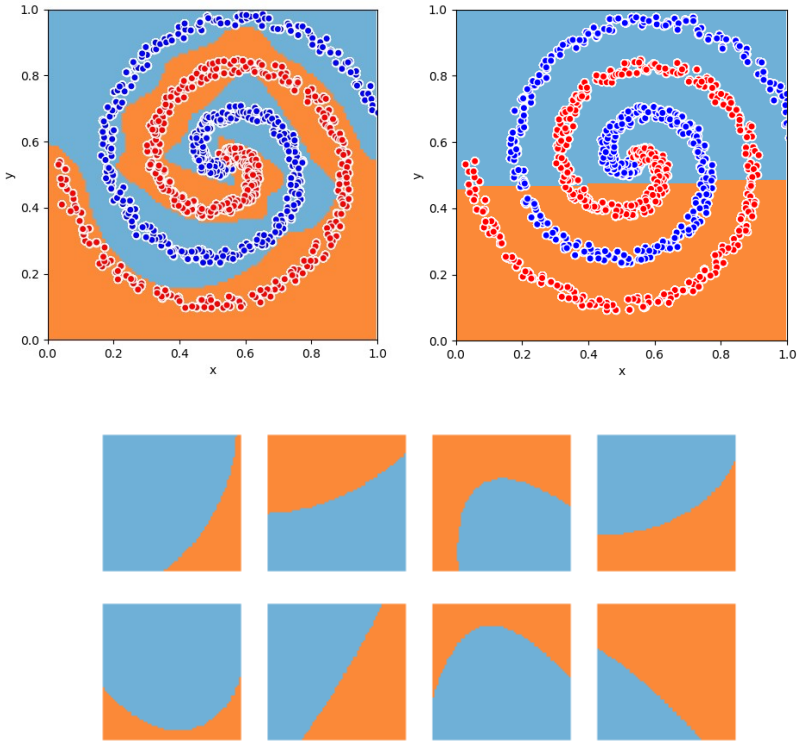


Fig. 10: Output response surfaces of a network comprised of 8 hidden nodes with polynomial expansions (up left) and without (up right). The output responses of each hidden node of (up left) is presented in (down). The training set (data points) are colored in red/blue, which indicate positive/negative class respectively while orange/light blue areas are the corresponding network predictions.

The baseline architecture of the suggested network, see Figure 11, is based on the PointNet architecture. Our main contribution is the addition of polynomial functions as part of the input domain. The addition of point coordinate powers to the input is a simple procedure that improves accuracy and decreases the run time during training and inference.

Performance Evaluation

Evaluation is performed on the ModelNet40 benchmark [36]. ModelNet40 is a synthetic dataset composed of Computer-Aided Design (CAD) models. ModelNet40 contains $12,311$ CAD models given as triangular meshes. Preprocessing of each triangular mesh as proposed in [26] yields 1024 points sampled from each triangular

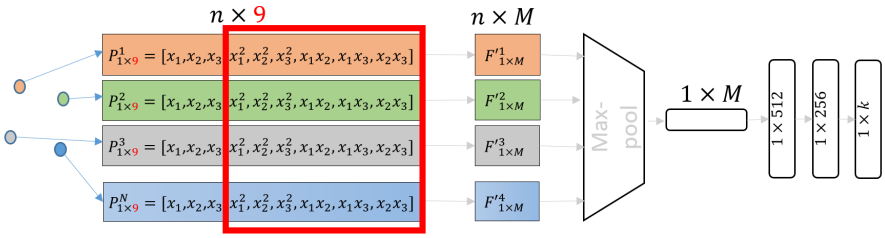


Fig. 11: Illustration of the proposed object classification architecture. The input of the network includes the point cloud coordinates as well as second order polynomial functions of these coordinates. It enables the network to efficiently learn the shape moments. The output is a probability distribution over the k classes, M is the number of features per point and n denotes the number of points.

mesh using the farthest point sampling (FPS) algorithm. Rotation by a random angle and additive noise are used for better data augmentation. The database contains samples from similar categories, like the flower-pot, plant and vase, for which separation is impossible even for a human observer.

M - Number of features per point	16	32	64	128	256	512
1st order	84.0	84.8	85.5	85.7	85.6	85.7
$1^{st} + 2^{nd}$ order	**84.6**	**85.6**	**86.5**	**86.5**	**86.7**	**87.2**

Table 1: Comparison of accuracy on ModelNet40 [36] (in %) between implicit 1st order input to our suggested network with implicit $1^{st} + 2^{nd}$ order input for different number of neurons in the feature layer. 1st order denotes only the point coordinates while $1^{st} + 2^{nd}$ order refer to the point coordinates concatenate with their polynomial functions as can been see in Figure 11.

We explore our implicit way of generating second order moments with respect to ModelNet40 database accuracy as a function of the number of nodes in the feature layer, see Table 1. The results show that there is a strong relation between the accuracy and the additional inputs that simplify the realization of geometric moments by the network. Our network has better accuracy percentage than the network without second order polynomials for all tested number of neurons in the feature layer. The only difference between the networks is the polynomial functions in the input domain.

A direct comparison with PointNet was also conducted and the experimental results confirm the superiority of such design. Our design is similar to that of PointNet in term of number of feature layers, however we consider also the polynomial expansions of the point coordinates. Table 2 presents a comparison with PointNet in term of accuracy, inference time and memory. The computational requirements are with respect to the number of the network's parameters (memory) and with respect to the

run-time required by the models for computing the predictions. Our results show that adding polynomial functions to input improve the classification performance, and in the same time lead to computational and memory efficiency.

	Memory (MB)	Inference Time (msec)	Overall Accuracy (%)
PointNet (baseline)	20	5.1	87.9
PointNet	40	5.6	89.2
Our Method	**20**	**5.1**	**89.6**

Table 2: Comparison of time and space complexity. Memory is the size of the model in mega-byte (MB) and the inference run-time was measured in milli-seconds (msec). Overall Accuracy stands for the suggested network accuracy on ModelNet40.

In conclusion, we proposed to feed the input domain with polynomial functions that represent geometric moments for 3D object classification. We combine an established method from the geometry processing literature with a neural network model to reinforce the model's capacity with a geometric concept. It is vital to develop methods which combine proven geometrical methods with deep neural networks to leverage their ability to generalize and produce more accurate models.

4 Conclusion

This chapter discusses the synergy between the principled axiomatic computations of geometric invariants and modern learning mechanisms like deep learning. Our attempt is to discover the space of computational methods that smoothly integrate principled axiomatic computation (geometric invariants) and data dependant models of deep learning. To this aim we demonstrated both ends of the coin: superior numerical geometry from learning and also improved deep learning using geometry. First, we have demonstrated a method to learn geometric invariants of planar curves. Using just positive and negative examples of transformations, we show that a convolutional neural network is able to train for an invariant which is numerically more robust as compared to differential and integral invariants. Second, we combined a geometric understanding about the ingredients required to construct compact shape signatures with neural networks that operate on clouds of points to leverage the network's abilities to cope with the problem of rigid objects classification. We demonstrated that lifting the input coordinates of points allowed the network to classify the objects with better efficiency and accuracy compared to previous methods that operate in the same domain. We believe that the ideas introduced in this chapter could be applied in other fields where geometric analysis is involved and could improve the ability of neural networks to efficiently and accurately handle geometric structures.

References

1. Ackerman, M.: Sophus Lie's 1884 Differential Invariant Paper. Math Sci Press (1976)
2. Atzmon, M., Maron, H., Lipman, Y.: Point convolutional neural networks by extension operators. ACM Trans. Graph. **37**(4), 71:1–71:12 (2018). DOI 10.1145/3197517.3201301. URL http://doi.acm.org/10.1145/3197517.3201301
3. Ben-Shabat, Y., Lindenbaum, M., Fischer, A.: 3dmfv: Three-dimensional point cloud classification in real-time using convolutional neural networks. IEEE Robotics and Automation Letters **3**(4), 3145–3152 (2018)
4. Bromley, J., Guyon, I., LeCun, Y., Säckinger, E., Shah, R.: Signature verification using a" siamese" time delay neural network. In: Advances in neural information processing systems, pp. 737–744 (1994)
5. Bronstein, A.M., Bronstein, M.M., Kimmel, R.: Numerical geometry of non-rigid shapes. Springer Science & Business Media (2008)
6. Bruckstein, A.M., Netravali, A.N.: On differential invariants of planar curves and recognizing partially occluded planar shapes. Annals of Mathematics and Artificial Intelligence **13**(3-4), 227–250 (1995)
7. Calabi, E., Olver, P.J., Shakiban, C., Tannenbaum, A., Haker, S.: Differential and numerically invariant signature curves applied to object recognition. International Journal of Computer Vision **26**(2), 107–135 (1998)
8. Carlevaris-Bianco, N., Eustice, R.M.: Learning visual feature descriptors for dynamic lighting conditions. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 2769–2776. IEEE (2014)
9. Cartan, E.: Geometry of Riemannian Spaces: Lie Groups; History, Frontiers and Applications Series, vol. 13. Math Science Press (1983)
10. Chopra, S., Hadsell, R., LeCun, Y.: Learning a similarity metric discriminatively, with application to face verification. In: 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), vol. 1, pp. 539–546. IEEE (2005)
11. Fidler, T., Grasmair, M., Scherzer, O.: Identifiability and reconstruction of shapes from integral invariants. Inverse Problems and Imaging **2**(3), 341–354 (2008)
12. Hadsell, R., Chopra, S., LeCun, Y.: Dimensionality reduction by learning an invariant mapping. In: 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), vol. 2, pp. 1735–1742. IEEE (2006)
13. Hong, B.W., Soatto, S.: Shape matching using multiscale integral invariants. IEEE transactions on pattern analysis and machine intelligence **37**(1), 151–160 (2014)
14. Hu, J., Lu, J., Tan, Y.P.: Discriminative deep metric learning for face verification in the wild. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1875–1882 (2014)
15. Hua, B.S., Tran, M.K., Yeung, S.K.: Pointwise convolutional neural networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 984–993 (2018)
16. Huang, Q., Wang, W., Neumann, U.: Recurrent slice networks for 3d segmentation of point clouds. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2626–2635 (2018)
17. Kanezaki, A., Matsushita, Y., Nishida, Y.: Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 5010–5019 (2018)
18. Kendall, D.G.: Shape manifolds, procrustean metrics, and complex projective spaces. Bulletin of the London Mathematical Society **16**(2), 81–121 (1984)
19. Kimmel, R.: Numerical geometry of images: Theory, algorithms, and applications. Springer Science & Business Media (2012)
20. Klokov, R., Lempitsky, V.: Escape from cells: Deep kd-networks for the recognition of 3d point cloud models. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 863–872 (2017)

21. Manay, S., Hong, B.W., Yezzi, A.J., Soatto, S.: Integral invariant signatures. In: European Conference on Computer Vision, pp. 87–99. Springer (2004)
22. Masci, J., Boscaini, D., Bronstein, M., Vandergheynst, P.: Geodesic convolutional neural networks on riemannian manifolds. In: Proceedings of the IEEE International Conference on Computer Vision Workshops, pp. 37–45 (2015)
23. Maturana, D., Scherer, S.: Voxnet: A 3d convolutional neural network for real-time object recognition. In: Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on, pp. 922–928. IEEE (2015)
24. Pai, G., Wetzler, A., Kimmel, R.: Learning invariant representations of planar curves. In: International Conference on Learning Representations (2017)
25. Pottmann, H., Wallner, J., Huang, Q.X., Yang, Y.L.: Integral invariants for robust geometry processing. Computer Aided Geometric Design **26**(1), 37–60 (2009)
26. Qi, C.R., Su, H., Mo, K., Guibas, L.J.: Pointnet: Deep learning on point sets for 3d classification and segmentation. Proc. Computer Vision and Pattern Recognition (CVPR), IEEE **1**(2), 4 (2017)
27. Qi, C.R., Yi, L., Su, H., Guibas, L.J.: Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In: Advances in Neural Information Processing Systems, pp. 5105–5114 (2017)
28. Riegler, G., Osman Ulusoy, A., Geiger, A.: Octnet: Learning deep 3d representations at high resolutions. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 3577–3586 (2017)
29. Su, B., Su, P.c.: Affine differential geometry. CRC Press (1983)
30. Su, H., Maji, S., Kalogerakis, E., Learned-Miller, E.: Multi-view convolutional neural networks for 3d shape recognition. In: Proceedings of the IEEE international conference on computer vision, pp. 945–953 (2015)
31. Sun, Y., Wang, X., Tang, X.: Deep learning face representation from predicting 10,000 classes. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1891–1898 (2014)
32. Taigman, Y., Yang, M., Ranzato, M., Wolf, L.: Deepface: Closing the gap to human-level performance in face verification. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1701–1708 (2014)
33. Wang, Y., Sun, Y., Liu, Z., Sarma, S.E., Bronstein, M.M., Solomon, J.M.: Dynamic graph cnn for learning on point clouds. arXiv preprint arXiv:1801.07829 (2018)
34. Weingarten, J.W., Gruener, G., Siegwart, R.: A state-of-the-art 3d sensor for robot navigation. In: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)(IEEE Cat. No. 04CH37566), vol. 3, pp. 2155–2160. IEEE (2004)
35. Weiss, I.: Noise-resistant invariants of curves. IEEE Transactions on Pattern Analysis and Machine Intelligence **15**(9), 943–948 (1993)
36. Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., Xiao, J.: 3d shapenets: A deep representation for volumetric shapes. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1912–1920 (2015)
37. Xiang, Y., Choi, W., Lin, Y., Savarese, S.: Data-driven 3d voxel patterns for object category recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1903–1911 (2015)
38. Xie, J., Fang, Y., Zhu, F., Wong, E.: Deepshape: Deep learned shape descriptor for 3d shape matching and retrieval. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1275–1283 (2015)
39. Yan, C., Xie, H., Yang, D., Yin, J., Zhang, Y., Dai, Q.: Supervised hash coding with deep neural network for environment perception of intelligent vehicles. IEEE transactions on intelligent transportation systems **19**(1), 284–295 (2017)
40. Yarotsky, D.: Error bounds for approximations with deep relu networks. Neural Networks **94**, 103–114 (2017)
41. Zhao, J., Xie, X., Xu, X., Sun, S.: Multi-view learning overview: Recent progress and new challenges. Information Fusion **38**, 43–54 (2017)